

Reviewer Pack — Minimal Rank of Noncommutative Tensor Products over Graphs v...

Entry d0b61fbda16b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-24 10:02:44 UTC

Minimal Rank of Noncommutative Tensor Products over Graphs vs Resolution Proof Length for Tseitin Formulas

Entry ID: d0b61fbda16b **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-24 10:02:44 UTC

1. Conjecture

Field A (mathematical branch): Noncommutative Geometry (Tensor Product Theory) **Field B** (complexity object): Complexity Theory: Resolution Proof Complexity for Tseitin Formulas

Statement:

[‘For a given graph G , the minimal rank of its noncommutative tensor product with itself, denoted as $\text{MinRank}(G \otimes G)$, is strictly greater than or equal to twice the Resolution proof length of the corresponding Tseitin formula on G , i.e., $\text{MinRank}(G \otimes G) \geq 2^{\text{ResolutionLength}(T_G)}$ for all graphs G .’, ‘This relationship holds with equality if and only if G has no cycles.’, ‘For graphs G with cycles, there exists a constant $c > 0$ such that $\text{MinRank}(G \otimes G) \geq 2^c * \text{ResolutionLength}(T_G)$.’]

Rationale (proposer’s reasoning):

[‘The noncommutative tensor product of graphs may capture higher-dimensional interactions between vertices, which are not visible in the original graph structure. This could potentially expose deeper complexity-theoretic properties.’, ‘Previous work has shown that certain graph invariants can be related to complexity measures for Tseitin formulas. It is plausible that noncommutative geometry provides a new angle on this relationship.’, ‘The proposed conjecture aims to explore the connection between noncommutative tensor products and Resolution proof length, providing a novel framework for understanding the computational hardness of Tseitin formulas.’]

Taxonomy category: TSEITIN_RES (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 05b3d881ebe8582b

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For each graph G with n vertices ($n \leq 40$), if $\text{MinRank}(G \otimes G) / \text{ResolutionLength}(T_G) \geq 2$, then the conjecture is supported; otherwise, it is falsified.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def min_rank(graph):
    n = len(graph)
    A = [[graph[i][j] * graph[k][l] for l in range(n)] for k in range(n) for j in range(n)]

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        rank = 0
        for i in range(min(m, n)):
            if A[i][i] == 0:
                for j in range(i + 1, m):
                    if A[j][i] != 0:
                        A[i], A[j] = A[j], A[i]
                        break
                else:
                    continue
            pivot = Fraction(A[i][i])
            for j in range(n):
                A[i][j] /= pivot
            for j in range(m):
                if j != i and A[j][i] != 0:
                    factor = -A[j][i]
                    for k in range(n):
                        A[j][k] += factor * A[i][k]
            rank += 1
        return rank

    return gaussian_elimination(A)

def resolution_length(graph):
```

```

n = len(graph)
clauses = []

def add_clause(clause):
    clauses.append(clause)

for i in range(n):
    for j in range(i + 1, n):
        if graph[i][j] == 1:
            add_clause([-i - 1, -j - 1])
            add_clause([i + 1, j + 1])
            add_clause([-i - 1, j + 1])
            add_clause([i + 1, -j - 1])

def is_satisfiable():
    stack = []
    assignment = {}

    def backtrack():
        if len(stack) == len(clauses):
            return True
        literal = next((lit for lit in range(1, n + 1) if lit not in assignment and -lit not in assignment))
        if literal is None:
            return False

        stack.append(literal)
        assignment[literal] = True
        while stack:
            lit = stack[-1]
            satisfied = any(any(clause[i] == 0 for i, val in enumerate(assignment) if val) for clause in clauses)
            if not satisfied:
                del assignment[lit]
                stack.pop()
                if -lit in assignment:
                    del assignment[-lit]
                else:
                    return backtrack()
            else:
                break
        return True

    return backtrack()

return len(clauses) if is_satisfiable() else 0

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    graph = [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]

    MinRank_G_tensor_G = min_rank(graph) ** 2
    ResolutionLength_T_G = resolution_length(graph)

    metric_value = MinRank_G_tensor_G / ResolutionLength_T_G if ResolutionLength_T_G > 0 else float('inf')
    conjecture_holds = metric_value >= 2
    counterexample = "" if conjecture_holds else f"Graph with n={n}, MinRank(G ⊗ G)={MinRank_G_tensor_G}, ResolutionLength(T_G)={ResolutionLength_T_G}"

```

```

    return {
        "metric_name": "MinRank( $G \otimes G$ ) / ResolutionLength( $T_G$ )",
        "metric_value": metric_value,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if len(sys.argv) > 1 else list(range(2, 30))

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"Graph with n={results[0]['instances_tested']}\", seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_79cf854f.py", line 118, in <module>

```

    result = run_trial(seed)
    ~~~~~
```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_79cf854f.py", line 97, in run_trial

```

    MinRank_G_tensor_G = min_rank(graph) ** 2
    ~~~~~
```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_79cf854f.py", line 20, in min_rank

```

    A = [[graph[i][j] * graph[k][l] for l in range(n)] for k in range(n) for j in range(n)]
        ^
```

NameError: name 'i' is not defined. Did you mean: 'id'?

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means that the pre-registered support condition could not be unambiguously met. | next: Debug the test code to ensure it runs successfully and check if the pre-registered support condition is met.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12758
2	propose	ollama_remote	glm4:latest	0	0	16314
3	preregistration	ollama_remote	glm4:latest	0	0	5468
4	novelty	ollama_remote	glm4:latest	0	0	5016
5	novelty	ollama_remote	glm4:latest	0	0	5287
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21691
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12586
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10748
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13200
10	judge	ollama_remote	glm4:latest	0	0	10577

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 113645 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/d0b61fbda16b.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/d0b61fbda16b.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/d0b61fbda16b.tar.gz (if generated)